توثيق مشروع

# AgriFresh Data Warehouse

Snowflake + dbt | RAW → Bronze → Silver → Gold

---

بناء مستودع بيانات من 13 ملف CSV: تحميل، تنظيف، اختبار، ونمذجة Star Schema

تاريخ الإصدار: 24 أكتوبر 2026

# المحتويات

1. نظرة عامة على المشروع
2. البيانات المصدر
3. مشاكل جودة البيانات
4. تجهيز Snowflake
5. تحميل البيانات (RAW)
6. مشروع dbt
7. طبقة Bronze
8. طبقة Silver
9. طبقة Gold (Star Schema)
10. الاختبارات (Tests)
11. التشغيل والنتائج
12. دليل التشغيل (Runbook)
13. أسئلة مفتوحة للبيزنس
ملحق أ: أعمدة الملفات المصدر
ملحق ب: هيكل ملفات المشروع

# 1. نظرة عامة على المشروع

المشروع بيبني **Data Warehouse** لشركة AgriFresh، وهي شركة توزيع خضار وفاكهة شغالة في **مصر والسعودية والإمارات**. الداتا جاية من 13 ملف CSV بتغطي الفترة من يناير 2024 لسبتمبر 2026: المبيعات، والمشتريات من المزارع، والشحن، والمدفوعات، والبيانات المرجعية.

## الهدف

- تجميع الداتا في مكان واحد (**Snowflake**) بشكل منظم وقابل للتتبع.
- تنظيف الداتا وتوحيدها، والتأكد من جودتها بـ **tests** آلية.
- تقديم الداتا في شكل **Star Schema** جاهز للتقارير والـ dashboards.

## الأدوات

| الأداة | الدور |
|---|---|
| Snowflake | تخزين الداتا وتنفيذ الـ queries (الـ compute والـ storage) |
| dbt (data build tool) | كتابة التحويلات كـ SQL، وترتيب تنفيذها، واختبارها، وتوثيقها |
| dbt_utils | مكتبة tests ودوال جاهزة (date spine، surrogate keys، ...) |
| Python 3.12 venv | بيئة تشغيل dbt على الجهاز (dbt-snowflake 1.12.4، dbt-core 1.12.6) |

## المعمارية (Architecture)

```
13 CSV files
   |   PUT + COPY INTO
   v
RAW      13 tables   all columns VARCHAR, + _source_file, _loaded_at
   |   dbt
   v
BRONZE   13 views    1:1 copy of RAW, no business logic
   |   dbt (cleaning)
   v
SILVER   14 tables   typed, cleaned, deduplicated, tested
   |   dbt (modelling)
   v
GOLD      5 tables   star schema: 3 dimensions + 2 facts
```

## ملخص الأرقام

| البند | القيمة |
|---|---|
| ملفات مصدر | 13 ملف CSV |
| صفوف RAW | 84,718 |
| Models في dbt | الإجمالي 32 (Bronze: 13، Silver: 14، Gold: 5) |
| Data tests | 142 |
| نتيجة آخر dbt build | PASS = 173، WARN = 1، ERROR = 0 |
| زمن الـ build الكامل | حوالي 16 ثانية |
| تكلفة التشغيل (آخر 24 ساعة) | حوالي 0.13 credit |

## 2. البيانات المصدر

الملفات نوعين: **Dimensions** (بيانات مرجعية بتوصف الكيانات) و **Facts** (معاملات وأحداث). التفاصيل الكاملة لكل عمود في ملحق أ.

### الملفات

| الملف | النوع | المحتوى | الصفوف |
|---|---|---|---|
| `regions.csv` | Dimension | المناطق الجغرافية (6 مناطق في 3 دول) وعملة كل منطقة | 6 |
| `product_categories.csv` | Dimension | فئات المنتجات، وفيها علاقة أب وابن | 6 |
| `warehouses.csv` | Dimension | مراكز التوزيع وسعتها | 5 |
| `farms.csv` | Dimension | المزارع الموردة والشهادات | 20 |
| `products.csv` | Dimension | المنتجات: SKU، التكلفة، سعر البيع | 40 |
| `employees.csv` | Dimension | فريق المبيعات والمديرين | 27 |
| `customers.csv` | Dimension | العملاء (B2B)، وفيه أكتر من نسخة لنفس العميل | 308 |
| `fx_rates.csv` | Reference | أسعار الصرف الشهرية للدولار | 132 |
| `purchase_orders.csv` | Fact | استلام البضاعة من المزارع | 3,500 |
| `sales_orders.csv` | Fact | رأس أوردر البيع | 12,000 |
| `sales_order_lines.csv` | Fact | بنود الأوردرات | 48,400 |
| `shipments.csv` | Fact | الشحنات (شحنة لكل أوردر) | 11,158 |
| `payments.csv` | Fact | مدفوعات العملاء | 9,116 |

### العلاقات بين الجداول

```
regions            <- warehouses, farms, customers, employees
product_categories <- products            (+ self parent_category_id)
employees          <- customers.account_manager_id, sales_orders.sales_rep_id
                      (+ self manager_id)
sales_orders       -> customers, employees, warehouses
sales_order_lines  -> sales_orders, products
shipments          -> sales_orders   (1 : 1)
payments           -> sales_orders
purchase_orders    -> farms, products, warehouses
fx_rates           <- sales_orders.currency_code + month(order_date)
```

## 3. مشاكل جودة البيانات

قبل كتابة أي كود، اتعمل **profiling** لكل عمود: عدد القيم الفاضية، والقيم المتكررة، وأشكال القيم، والقيم الشاذة، وسلامة العلاقات. ده اللي طلع، والقرار اللي اتاخد في كل مشكلة:

| الجدول | المشكلة | مثال / حجم | القرار |
|---|---|---|---|
| `customers` | نفس العميل موجود أكتر من مرة | 8 عملاء (308 صف لـ 300 عميل) | Silver: آخر نسخة بـ updated_at. التاريخ الكامل في SCD2 |
| `customers` | حروف كبيرة وصغيرة مخلوطة في segment | `supermarket` و `Supermarket` (12 قيمة بدل 6) | INITCAP(TRIM()) |
| `customers` | email فاضي | 24 صف | يفضل NULL |
| `products` | أسماء CAPS وفيها مسافات | `ONION WHITE` | INITCAP(TRIM()) |
| `farms` | كلمة None مكان القيمة الفاضية | 4 مزارع | تتحول لـ NULL، ويتضاف is_certified |
| `product_categories` | رقم مكتوب كـ float | `1.0` بدل 1 | تحويل لـ integer |
| `fx_rates` | سعر الدولار للدولار مش 1 | `1.000987` | يتثبت على 1 |
| `sales_order_lines` | منتج مش موجود في جدول المنتجات | product_id = 9999 (سطر واحد) | يتشال من Silver |
| `sales_order_lines` | كميات سالبة | 84 سطر | تفضل، وعليها flag، والـ test نوعه warn |
| `payments` | مبالغ سالبة | 9 دفعات | تفضل، وعليها flag اسمه is_refund |
| `payments` | تاريخ في المستقبل | 256 دفعة لحد 30-11-2026 | تفضل، وعليها flag اسمه is_future_dated |
| `shipments` | delivered_at فاضي | 747 شحنة (كلها حالتها SHIPPED) | منطقي، ويتضاف is_delivered |

> **قاعدة عامة:** أي حاجة واضح إنها غلط تقني بتتصلح. وأي حاجة محتاجة قرار من البيزنس **مابتتمسحش**، بيتحط عليها flag والبيزنس يقرر.

# 4. تجهيز Snowflake

الملف: `snowflake/01_setup.sql`. اتشغل مرة واحدة بالـ roles `SYSADMIN` و `SECURITYADMIN`.

## الكائنات (Objects)

| الكائن | الاسم | الوصف |
|---|---|---|
| Warehouse | `AGRI_WH` | حجم XSMALL، و AUTO_SUSPEND بعد 60 ثانية، فمش بيصرف credits وهو واقف |
| Database | `AGRIFRESH` | قاعدة البيانات الرئيسية |
| Schema | `RAW` | جداول الهبوط (landing)، وكل الأعمدة نص |
| Schema | `BRONZE` | views بتعملها dbt، نسخة 1:1 من RAW |
| Schema | `SILVER` | جداول نضيفة ومختبرة |
| Schema | `GOLD` | Star Schema للتقارير |

## الأدوار والصلاحيات (Roles)

الصلاحيات اتقسمت بمبدأ **أقل صلاحية لازمة** (least privilege):

| Role | الصلاحيات | بيستخدمه مين |
|---|---|---|
| `LOADER` | كل الصلاحيات على RAW بس | سكريبت التحميل |
| `TRANSFORMER` | قراءة من RAW، وإنشاء في BRONZE و SILVER و GOLD | dbt |

## طريقة الاتصال: Key-Pair Authentication

بدل الباسورد، الاتصال بيستخدم زوج مفاتيح RSA:

1. اتعمل زوج مفاتيح على الجهاز: المفتاح الخاص `~/.snowflake/agrifresh_rsa_key.p8` (صلاحيته 600، ومابيخرجش من الجهاز)، والمفتاح العام `agrifresh_rsa_key.pub`.
2. المفتاح العام اتسجل على اليوزر في Snowsight بـ `ALTER USER ... SET RSA_PUBLIC_KEY`.
3. dbt والسكريبتات بيتصلوا بالمفتاح الخاص، فالباسورد مش مكتوب في أي ملف.

> **ملحوظة:** الاتصال لازم يكون بالـ **login name** (`<LOGIN_NAME>`) مش باسم اليوزر (`<USER_NAME>`)، لأن الاتصال بالاسم بيطلع `JWT token is invalid`.

| الإعداد | القيمة |
|---|---|
| Account identifier | `<ORGNAME-ACCOUNTNAME>` |
| Region | `AWS eu-central-2` |
| Login name | `<LOGIN_NAME>` |
| Private key | `~/.snowflake/agrifresh_rsa_key.p8` |

## 5. تحميل البيانات (RAW)

الملف: `snowflake/02_load_raw.sql`. اتشغل بالـ role `LOADER`.

1. **File Format** اسمه `CSV_FORMAT`: أول سطر header، والخانة الفاضية معناها NULL.
2. **Stage** اسمه `AGRIFRESH_STAGE`: مكان مؤقت جوه Snowflake، والملفات بتترفع عليه بأمر `PUT` (مضغوطة gzip).
3. **13 جدول** في RAW، **وكل الأعمدة VARCHAR**، ومعاهم عمودين: `_source_file` (الصف جه منين) و `_loaded_at` (اتحمل امتى).
4. **COPY INTO** بينقل الداتا من الـ stage للجداول، و `ON_ERROR = ABORT_STATEMENT`.

> **ليه كل الأعمدة نص؟** لو عمود المفروض رقم وجاله `1.0` أو `None`، التحميل كان هيقع. فبنحمّل الداتا زي ما هي، والتحويل للأنواع الصح بيحصل في dbt، وهناك نقدر نختبره ونتحكم فيه.

```
PUT file:///Users/abdelrahmantarek/agrifresh_dwh/data/*.csv @AGRIFRESH_STAGE
    AUTO_COMPRESS = TRUE OVERWRITE = TRUE;

COPY INTO CUSTOMERS (CUSTOMER_ID, ..., _SOURCE_FILE, _LOADED_AT)
FROM (
  SELECT t.$1, ..., METADATA$FILENAME, CURRENT_TIMESTAMP()
  FROM @AGRIFRESH_STAGE/customers.csv t
)
ON_ERROR = 'ABORT_STATEMENT';
```

**النتيجة:** الـ 13 ملف اتحملوا، و**عدد الصفوف في كل جدول بيساوي عدد الصفوف في الـ CSV بالظبط**، ومفيش ولا خطأ. الإجمالي 84,718 صف.

# 6. مشروع dbt

dbt بيخليك تكتب كل تحويل كملف `SELECT`. هو بيعرف ترتيب التنفيذ لوحده من `ref()` و `source()`، وبيعمل الجداول في Snowflake، وبيشغل الـ tests، وبيطلّع توثيق كامل. كل ده بأمر واحد: `dbt build`.

### ملفات الإعداد

| الملف | الدور |
|---|---|
| `dbt_project.yml` | كل فولدر بيروح أنهي schema، وبيتعمل view ولا table، وعليه أنهي tag |
| `profiles.yml` | بيانات الاتصال بـ Snowflake (key-pair)، والقيم بتيجي من environment variables |
| `packages.yml` | مكتبة dbt_utils (من 1.1.0 لأقل من 2.0.0) |
| `macros/generate_schema_name.sql` | بيخلي اسم الـ schema هو SILVER بدل الاسم الافتراضي SILVER_SILVER |
| `macros/cleaning.sql` | دوال تنظيف بنعيد استخدامها في كل الموديلات |

### الـ Materialization لكل طبقة

| الطبقة | Schema | النوع | Tag |
|---|---|---|---|
| Bronze | `BRONZE` | `view` | `bronze` |
| Silver | `SILVER` | `table` | `silver` |
| Gold | `GOLD` | `table` | `gold` |

### Macros التنظيف

كلها بتستخدم `try_to_*` بدل `CAST`، عشان القيمة البايظة تبقى NULL بدل ما الـ query كلها تقع، وبعدين الـ tests تمسكها.

| Macro | بيعمل إيه |
|---|---|
| `clean_string(col)` | TRIM، والقيم الفاضية و None و NULL و N/A تبقى NULL |
| `yn_to_bool(col)` | Y تبقى true، و N تبقى false |
| `to_int(col)` | رقم صحيح، حتى لو مكتوب 1.0 |
| `to_dec(col, scale)` | رقم عشري |
| `to_date(col)` | تاريخ بصيغة YYYY-MM-DD |
| `to_ts(col)` | timestamp بصيغة YYYY-MM-DD HH24:MI:SS |

## 7. طبقة Bronze

13 view (`brz_customers`، `brz_sales_orders`، ...)، وكل واحد فيهم `SELECT` صريح لكل أعمدة جدول RAW المقابل، **من غير أي تعديل**.

### ليه طبقة من غير تعديل؟

- **نقطة بداية ثابتة** جوه dbt، فالـ lineage بيبان من المصدر لآخر جدول.
- لو المصدر اتغير (مثلاً اسم عمود اتغير)، التعديل بيبقى في **مكان واحد بس**.
- عملناها **views**، فمش بتاخد مساحة تخزين.

الـ sources متعرفة في `models/bronze/_sources.yml`، وعلى كل Bronze view فيه test اسمه `not_null` على الـ primary key.

## 8. طبقة Silver

هنا التنظيف الحقيقي. كل موديل بيعمل 3 حاجات: **يحوّل الأنواع**، و**يصلح المشاكل**، و**يحط flags** على الحاجات اللي محتاجة قرار. وكمان كل موديل بيعمل dedup بـ `QUALIFY ROW_NUMBER()` على الـ primary key، عشان لو نفس الداتا اتحملت مرتين.

| Model | التنظيف | أعمدة جديدة | الصفوف |
|---|---|---|---|
| `slv_regions` | تحويل الأنواع، والأكواد UPPER | | 6 |
| `slv_product_categories` | parent 1.0 يبقى integer | `is_top_level` | 6 |
| `slv_warehouses` | Y/N يبقى boolean | | 5 |
| `slv_farms` | None يبقى NULL، والإيميل lower | `is_certified` | 20 |
| `slv_products` | INITCAP و TRIM للأسماء | `unit_margin_usd, margin_pct` | 40 |
| `slv_employees` | تحويل التواريخ | `is_active` | 27 |
| `slv_customers` | آخر نسخة لكل عميل، و segment بـ INITCAP | | 300 |
| `slv_customers_history` | كل النسخ (SCD Type 2) | `valid_from, valid_to, is_current` | 308 |
| `slv_fx_rates` | USD→USD يبقى 1 | | 132 |
| `slv_purchase_orders` | تحويل الأنواع | `quantity_accepted, reject_rate, accepted_cost_usd, harvest_to_receipt_days` | 3,500 |
| `slv_sales_orders` | الـ status بـ UPPER | `order_month` | 12,000 |
| `slv_sales_order_lines` | حذف product 9999 | `is_negative_quantity, gross_amount_usd, discount_amount_usd, net_amount_usd` | 48,399 |
| `slv_shipments` | delivered_at الفاضي يفضل NULL | `is_delivered, transit_hours, is_on_time` | 11,158 |
| `slv_payments` | تحويل الأنواع | `is_refund, is_future_dated` | 9,116 |

### مثال: slv_customers

```
select
    {{ to_int('customer_id') }}             as customer_id,
    initcap({{ clean_string('segment') }})  as segment,
    {{ to_dec('credit_limit_usd', 2) }}     as credit_limit_usd,
    lower({{ clean_string('email') }})      as email,
    {{ to_ts('updated_at') }}               as updated_at
from {{ ref('brz_customers') }}
qualify row_number() over (
    partition by customer_id
    order by {{ to_ts('updated_at') }} desc
) = 1
```

### تاريخ العملاء (SCD Type 2)

بدل ما النسخ القديمة تترمي، `slv_customers_history` بيحتفظ بكل نسخة ومعاها فترة صلاحيتها. مثلاً عميل 10:

| is_current | valid_to | valid_from | credit_limit_usd | customer_id |
|---|---|---|---|---|
| false | 2025-10-01 | 2025-09-01 | 5,000 | 10 |
| true | NULL | 2025-10-01 | 10,000 | 10 |

**ليه مش dbt snapshot؟** الـ snapshot بيشوف آخر نسخة بس كل مرة بيشتغل. والنسخ القديمة هنا موجودة أصلاً في نفس الملف، فاستخدمنا `LEAD()` عشان نبني التاريخ منها مباشرة.

# 9. طبقة Gold (Star Schema)

الـ Silver نضيفة، لكن لسه شكلها شكل نظام التشغيل: جداول كتير مربوطة ببعض. الـ Gold بتعيد ترتيبها في **Star Schema**: جداول **Facts** فيها الأرقام والأحداث، وحواليها **Dimensions** فيها الوصف اللي بنقطّع بيه الأرقام.

```
                    dim_date
                       |
dim_customer ---- fact_sales ---- dim_product ---- fact_purchases
                                                        |
                                                    dim_date
```

## Dimensions

| الجدول | الـ Grain | الصفوف | المحتوى |
|---|---|---|---|
| `dim_date` | يوم | 1,826 | من 2023 لـ 2027، و date_key بشكل YYYYMMDD، والسنة والربع والشهر والأسبوع. الويك إند جمعة وسبت |
| `dim_customer` | عميل | 300 | آخر نسخة (SCD1)، ومعاها المنطقة والدولة والعملة واسم الـ account manager |
| `dim_product` | منتج | 40 | الـ category والـ parent category، والتكلفة والسعر والـ margin |

## Facts

| الجدول | الـ Grain | الصفوف | Measures أساسية |
|---|---|---|---|
| `fact_sales` | سطر أوردر | 48,399 | `gross_amount_usd, discount_amount_usd, net_amount_usd, cost_amount_usd, gross_margin_usd, net_amount_local, fx_rate_to_usd, is_revenue_recognized` |
| `fact_purchases` | أمر شراء | 3,500 | `quantity_received, quantity_rejected, quantity_accepted, received_cost_usd, accepted_cost_usd, rejected_cost_usd, reject_rate, harvest_to_receipt_days` |

### قرارات التصميم

- **Surrogate keys** (`customer_key` و `product_key`) معمولة بـ `dbt_utils.generate_surrogate_key`، و `date_key` رقم YYYYMMDD.
- **العملة:** الأسعار في المصدر بالدولار. `net_amount_local` محسوبة بسعر صرف شهر الأوردر (المحلي = الدولار ÷ السعر).
- `is_revenue_recognized` قيمته true للأوردرات اللي حالتها DELIVERED أو SHIPPED بس.
- `warehouse_id` و `sales_rep_id` و `farm_id` فضلوا IDs في الـ facts، لأن مفيش dims ليهم دلوقتي.

### أمثلة من النتائج

صافي المبيعات المعترف بيها والـ margin، حسب السنة والدولة (بالدولار):

| السنة | الدولة | صافي المبيعات | الـ Margin | نسبة الـ Margin |
|---|---|---|---|---|
| 2024 | مصر | 233,289 | 82,028 | 35.2% |
| 2024 | السعودية | 154,060 | 54,299 | 35.2% |
| 2024 | الإمارات | 88,771 | 31,416 | 35.4% |

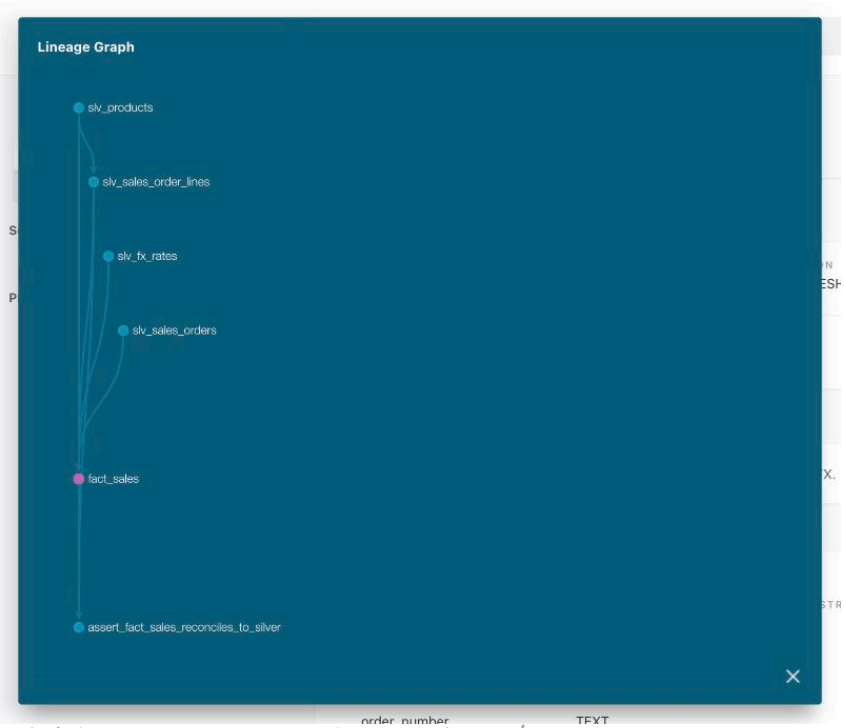
| السنة | الدولة | صافي المبيعات | الـ Margin | نسبة الـ Margin |
|---|---|---|---|---|
| 2025 | مصر | 575,555 | 200,402 | 34.8% |
| 2025 | السعودية | 448,714 | 155,116 | 34.6% |
| 2025 | الإمارات | 195,082 | 67,370 | 34.5% |
| 2026 | مصر | 901,665 | 307,588 | 34.1% |
| 2026 | السعودية | 586,716 | 202,939 | 34.6% |
| 2026 | الإمارات | 274,933 | 95,837 | 34.9% |

بيانات 2026 لحد سبتمبر بس.

### مثال Query

```
SELECT d.year, c.country, ROUND(SUM(f.net_amount_usd)) AS net_usd
FROM AGRIFRESH.GOLD.FACT_SALES f
JOIN AGRIFRESH.GOLD.DIM_DATE d     ON f.order_date_key = d.date_key
JOIN AGRIFRESH.GOLD.DIM_CUSTOMER c ON f.customer_key  = c.customer_key
WHERE f.is_revenue_recognized
GROUP BY 1, 2
ORDER BY 1, 2;
```

## Lineage: fact_sales جاية منين




Lineage graph من dbt docs: أربع جداول Silver بتغذي fact_sales، وبعدها test المطابقة.

## 10. الاختبارات (Tests)

كل test عبارة عن query بتدور على صفوف غلط، ولو لقت **ولا صف واحد** الـ test بيقع. المشروع فيه **142 اختبار** موزعين على الطبقات التلاتة.

| النوع | بيتأكد من إيه | مثال |
|---|---|---|
| `unique / not_null` | الـ primary key سليم | مفيش order_id متكرر أو فاضي |
| `relationships` | كل foreign key ليه أصل | كل customer_id في الأوردرات موجود في جدول العملاء |
| `accepted_values` | القيم من قايمة معروفة | order_status واحدة من الـ 5 حالات بس |
| `expression_is_true` | قاعدة منطقية | quantity_rejected ≤ quantity_received |
| `accepted_range` | القيمة في نطاق | discount_pct بين 0 و 1 |
| `unique_combination` | مفتاح مركب | rate_month + from_currency + to_currency |

### Custom tests في فولدر tests

| Test | القاعدة |
|---|---|
| `assert_delivered_after_shipped` | تاريخ التسليم مش قبل تاريخ الشحن |
| `assert_payment_not_before_order` | الدفع مش قبل تاريخ الأوردر |
| `assert_shipped_orders_have_shipment` | كل أوردر حالته SHIPPED أو DELIVERED أو RETURNED ليه شحنة |
| `assert_fact_sales_reconciles_to_silver` | fact_sales فيها نفس عدد السطور ونفس إجمالي net_amount_usd اللي في Silver، فمفيش سطر ضاع أو اتكرر |

**الـ warning الوحيد:** test الكميات السالبة (84 سطر) نوعه **warn** مش error. يعني بينبّه من غير ما يوقف الـ build، لحد ما البيزنس يقرر الكميات دي مرتجعات ولا غلط.

## 11. التشغيل والنتائج

| الخطوة | النتيجة |
|---|---|
| `01_setup.sql` | اتعملت كل الكائنات والصلاحيات من غير أخطاء |
| `02_load_raw.sql` | 13 جدول، و 84,718 صف، و 0 أخطاء |
| `dbt build` | كل الطبقات: ERROR = 0، WARN = 1، PASS = 173 من 174، في حوالي 16 ثانية |
| `dbt build -s tag:gold` | PASS = 43، من غير warnings |
| `dbt docs generate` | التوثيق والـ lineage اتعملوا |

الـ build اتشغل كذا مرة ورا بعض، من الجهاز ومن البيئة المؤقتة، وطلع نفس النتيجة كل مرة. يعني المشروع **reproducible**، وإعادة التشغيل مابتكررش الداتا.

### عدد الصفوف في كل طبقة

| Schema | النوع | العدد | الصفوف |
|---|---|---|---|
| RAW | tables | 13 | 84,718 |

| Schema | النوع | العدد | الصفوف |
|---|---|---|---|
| BRONZE | views | 13 | نفس RAW |
| SILVER | tables | 14 | 85,017 |
| GOLD | tables | 5 | 54,065 |

# 12. دليل التشغيل (Runbook)

### أول مرة على جهاز جديد

```
cd ~/agrifresh_dwh
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python dbt-snowflake
.venv/bin/dbt deps
```

**مهم:** الـ dbt المتسطب بـ Homebrew بيقع على Python 3.14، فلازم venv بـ Python 3.12.

### كل مرة تفتح terminal

```
cd ~/agrifresh_dwh
source .venv/bin/activate
export SNOWFLAKE_ACCOUNT=<ORGNAME-ACCOUNTNAME>
export SNOWFLAKE_USER=<LOGIN_NAME>
```

### الأوامر الأساسية

| الأمر | بيعمل إيه |
|---|---|
| `dbt debug` | يتأكد من الإعدادات والاتصال |
| `dbt build` | يبني كل الطبقات ويشغل كل الـ tests |
| `dbt build -s tag:silver` | طبقة واحدة بس |
| `dbt build -s +tag:gold` | الـ Gold ومعاها كل اللي قبلها |
| `dbt build -s slv_customers+` | موديل واحد وكل اللي بيعتمد عليه |
| `dbt docs generate && dbt docs serve` | التوثيق على http://localhost:8080 |

### تحميل داتا جديدة

1. حط ملفات الـ CSV الجديدة في `data/`.
2. شغّل `snowflake/02_load_raw.sql` من SnowSQL أو Snowflake CLI، لأن أمر PUT مابيشتغلش من Snowsight.
3. شغّل `dbt build`.

**ملحوظة:** Snowflake بيفتكر الملفات اللي اتحملت قبل كده، فـ `COPY INTO` مش هيحمّل نفس الملف مرتين. ولو عايز تعيد التحميل من الأول، استخدم `FORCE = TRUE` أو امسح الجدول الأول.

### تشغيل SQL في Snowsight

- زرار **Run** بيشغّل الـ statement اللي عليها الـ cursor بس. عشان تشغّل كل الـ worksheet، استخدم **Run All** (Cmd+Shift+Enter).
- اختار الـ role (`ACCOUNTADMIN` أو `TRANSFORMER`) والـ warehouse `AGRI_WH` من فوق.

# 13. أسئلة مفتوحة للبيزنس

| السؤال | الحجم | الوضع الحالي |
|---|---|---|
| الكميات السالبة في بنود الأوردرات: مرتجعات ولا غلط في الإدخال؟ | 84 سطر | موجودة، وعليها flag، والـ test نوعه warn |
| المدفوعات اللي تاريخها في المستقبل: شيكات مؤجلة ولا داتا غلط؟ | 256 دفعة | موجودة، وعليها is_future_dated |
| المبالغ السالبة في المدفوعات: refunds؟ | 9 دفعات | موجودة، وعليها is_refund |
| product_id 9999: سجل تجريبي المفروض يتمسح من المصدر؟ | سطر واحد | اتشال من Silver |

## خطوات مقترحة بعد كده

- تحويل `dim_customer` لـ SCD Type 2 باستخدام `slv_customers_history`.
- إضافة dims للمخازن والمزارع والموظفين.
- إضافة `fact_payments` و `fact_shipments` لتحليل التحصيل والتوصيل.
- تحويل الـ facts لـ **incremental models** لما حجم الداتا يكبر.
- حفظ المشروع في **git** وتشغيله أوتوماتيك (dbt Cloud أو GitHub Actions).

# ملحق أ: أعمدة الملفات المصدر

### regions.csv

| العمود | الوصف |
|---|---|
| region_id | PK |
| region_code | كود المنطقة (EG-CAI, SA-RIY, AE-DXB, ...) |
| region_name | اسم المنطقة |
| country | الدولة |
| currency_code | العملة: EGP / SAR / AED |

### product_categories.csv

| العمود | الوصف |
|---|---|
| category_id | PK |
| category_code | VEG, FRU, HRB, CIT, DAT, TUB |
| category_name | اسم الفئة |
| parent_category_id | الفئة الأب (فاضي للرئيسية، ومكتوب كـ 1.0) |

### warehouses.csv

| العمود | الوصف |
|---|---|
| warehouse_id | PK |
| warehouse_code | كود المخزن |
| warehouse_name | اسم المخزن |
| region_id | FK للمنطقة |
| capacity_tonnes | السعة بالطن |
| is_cold_storage | Y/N، تبريد |

### farms.csv

| العمود | الوصف |
|---|---|
| farm_id | PK |
| farm_code | كود المزرعة |
| farm_name | الاسم |
| region_id | FK للمنطقة |
| farm_size_feddan | المساحة بالفدان |
| certification | GlobalG.A.P. / Organic EU / None |
| contract_start_date | تاريخ بداية التعاقد |
| contact_email | إيميل التواصل (فاضي في صفين) |

### products.csv

| العمود | الوصف |
|---|---|
| product_id | PK (يبدأ من 1001) |
| sku | كود المنتج AF-xx-xxxx |
| product_name | الاسم |
| category_id | FK للفئة |
| unit_of_measure | KG / BUNCH / HEAD |
| shelf_life_days | مدة الصلاحية بالأيام |
| standard_cost_usd | التكلفة |
| list_price_usd | سعر البيع |
| is_organic | Y/N |
| is_active | Y/N |
| created_at | تاريخ الإنشاء |

### employees.csv

| العمود | الوصف |
|---|---|
| employee_id | PK (501 لـ 527) |
| full_name | الاسم |
| job_title | Account Executive / Regional Sales Manager |
| region_id | FK للمنطقة |
| manager_id | FK للمدير (self) |
| hire_date | تاريخ التعيين |
| termination_date | تاريخ إنهاء الخدمة (اتنين بس) |

### customers.csv

| العمود | الوصف |
|---|---|
| customer_id | PK (فيه تكرار) |
| customer_code | كود العميل |
| customer_name | الاسم |
| segment | / Supermarket / Hypermarket / Restaurant / Hotel / Online Grocer Wholesaler |
| region_id | FK للمنطقة |
| account_manager_id | FK للموظف |
| credit_limit_usd | حد الائتمان |
| payment_terms_days | مدة السداد (0 لـ 60 يوم) |

| العمود | الوصف |
|---|---|
| email | الإيميل |
| phone | التليفون |
| registered_at | تاريخ التسجيل |
| updated_at | آخر تعديل |

### fx_rates.csv

| العمود | الوصف |
|---|---|
| rate_month | أول يوم في الشهر |
| from_currency | EGP / SAR / AED / USD |
| to_currency | دايماً USD |
| rate | سعر التحويل للدولار |

### purchase_orders.csv

| العمود | الوصف |
|---|---|
| purchase_order_id | PK |
| po_number | رقم أمر الشراء |
| farm_id | FK للمزرعة |
| product_id | FK للمنتج |
| warehouse_id | FK للمخزن |
| harvest_date | تاريخ الحصاد |
| received_date | تاريخ الاستلام |
| quantity_received | الكمية المستلمة |
| quantity_rejected | الكمية المرفوضة |
| unit_cost_usd | تكلفة الوحدة |
| quality_grade | A / B / C |

### sales_orders.csv

| العمود | الوصف |
|---|---|
| order_id | PK |
| order_number | رقم الأوردر |
| customer_id | FK للعميل |
| sales_rep_id | FK للموظف |
| warehouse_id | FK للمخزن |
| order_date | تاريخ الأوردر |

| العمود | الوصف |
|---|---|
| requested_delivery_date | تاريخ التسليم المطلوب |
| order_status | PENDING / SHIPPED / DELIVERED / CANCELLED / RETURNED |
| currency_code | العملة |
| channel | B2B Portal / Sales Rep / EDI / Phone |
| created_at | وقت الإنشاء |

### sales_order_lines.csv

| العمود | الوصف |
|---|---|
| order_line_id | PK |
| order_id | FK للأوردر |
| product_id | FK للمنتج |
| quantity | الكمية (فيها 84 سالبة) |
| unit_price_usd | سعر الوحدة |
| discount_pct | الخصم (0 / 0.05 / 0.1 / 0.15) |

### shipments.csv

| العمود | الوصف |
|---|---|
| shipment_id | PK |
| order_id | FK للأوردر (1:1) |
| carrier | DHL / Aramex / SMSA / Bosta / Own Fleet |
| shipped_at | وقت الشحن |
| delivered_at | وقت التسليم (فاضي لو لسه في الطريق) |
| temperature_breach_flag | Y/N، تجاوز درجة الحرارة |
| freight_cost_usd | تكلفة الشحن |

### payments.csv

| العمود | الوصف |
|---|---|
| payment_id | PK |
| order_id | FK للأوردر |
| payment_date | تاريخ الدفع |
| amount_usd | المبلغ (فيه 9 سالبة) |
| payment_method | Bank Transfer / Card / Cash / Cheque |

# ملحق ب: هيكل ملفات المشروع

المسار: `~/agrifresh_dwh`

```
agrifresh_dwh/
├── data/                       13 source CSVs
├── snowflake/
│   ├── 01_setup.sql            warehouse, database, schemas, roles
│   └── 02_load_raw.sql         file format, stage, PUT, COPY INTO
├── macros/
│   ├── cleaning.sql            clean_string, yn_to_bool, to_int, ...
│   └── generate_schema_name.sql
├── models/
│   ├── bronze/                 _sources.yml, _bronze.yml, brz_*.sql (13)
│   ├── silver/                 _silver.yml, slv_*.sql (14)
│   └── gold/                   _gold.yml, dim_*.sql (3), fact_*.sql (2)
├── tests/                      4 custom SQL tests
├── dbt_project.yml
├── profiles.yml                key-pair auth via env vars
├── packages.yml                dbt_utils
└── README.md
```